



**ALLIANCE
UNIVERSITY**
Private University established in Karnataka State by Act No. 34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY

Bachelor of Computer Applications

Semester – V

Deep learning

Assignment No.: 6

Title: Neural Network Forward Propagation

Submitted By

Name: Dipin Roka

Reg. No.:2411021240007

Date:27/09/2026

[Github link](#)

Submitted To:

Faculty Name: Nagasri p

Department of Computer Application

Alliance University

Chandapura - Anekal Main Road, Anekal Bengaluru – 562106

Project 6: Simple Neural Network Forward Propagation

Dataset: Iris flower classification dataset

Goal: Understand how a neural network turns raw input features into a prediction by manually building the network and running **forward propagation** on the Iris dataset with NumPy.

This notebook covers:

- **Task 1:** Load and prepare the Iris dataset for classification.
- **Task 2:** Build a simple neural network and perform forward propagation.
- **Task 3:** Apply an activation function and display the predicted output.

Setup

Import the libraries we'll need. `numpy` does the math, `pandas` helps us look at the data, `matplotlib/seaborn` help us visualize it, and `scikit-learn` gives us an easy way to load the Iris dataset and split it into train/test sets.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split from sklearn.preprocessing import
StandardScaler

np.random.seed(42) # for reproducible results
%matplotlib inline
```

Task 1: Load and Prepare the Iris Dataset

We'll load the Iris dataset using `scikit-learn` (it ships with the same data as the classic Iris CSV, including the one linked in the assignment: <https://gist.github.com/curran/a08a1080b88344b0c8a7>).

The dataset has:

- **4 input features:** sepal length, sepal width, petal length, petal width (all in cm)
- **1 target:** the species of iris flower (Setosa, Versicolor, or Virginica)

```
iris = load_iris()

X = iris.data # shape (150, 4) -> the 4 input features
```

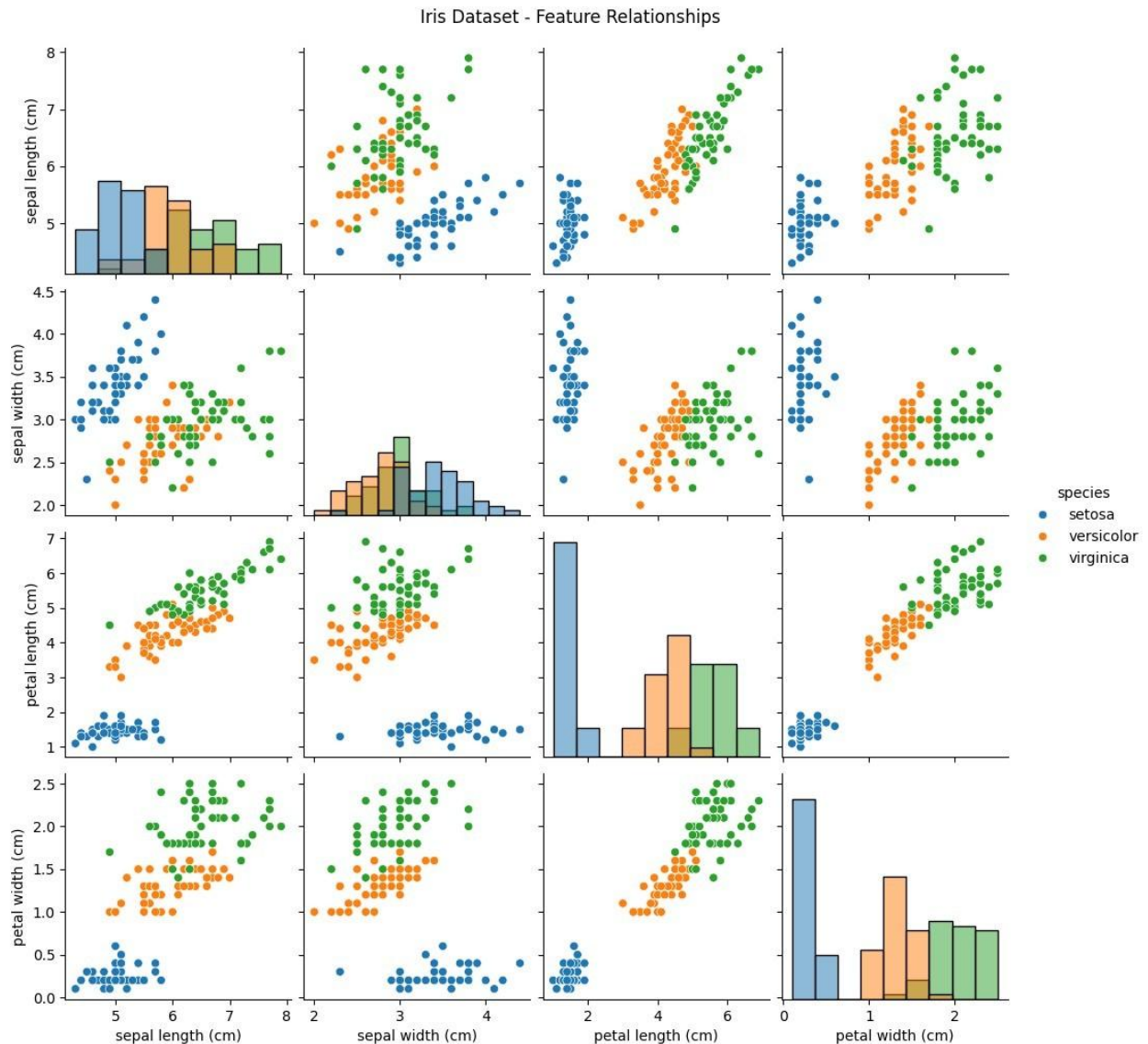
```

y = iris.target                                # shape (150,)  -> class labels:
0, 1,
2
feature_names = iris.feature_names
target_names = iris.target_names

df = pd.DataFrame(X, columns=feature_names)
df['species'] = pd.Categorical.from_codes(y, target_names)

print("Dataset shape:", X.shape)
print("Classes:", target_names)
df.head()
Dataset shape: (150,
4)
Classes:      ['setosa'      'versicolor'
'virginica']
{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\n    {\n      \"column\": \"sepal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n        \"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n          4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sepal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.435866284936698,\n        \"min\": 2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n        \"samples\": [\n          2.3,\n          4.0,\n          3.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.7652982332594667,\n        \"min\": 1.0,\n        \"max\": 6.9,\n        \"num_unique_values\": 43,\n        \"samples\": [\n          6.7,\n          3.8,\n          3.7\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.7622376689603465,\n        \"min\": 0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n        \"samples\": [\n          0.2,\n          1.2,\n          1.3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"species\",\n      \"properties\": {\n        \"dtype\": \"category\",

```

Prepare the data for the neural network

1. **Split** into training and test sets.
2. **Scale/standardize** the features so each one has mean 0 and standard deviation 1. Neural networks train and behave much better on standardized inputs.
3. **One-hot encode** the labels, since our output layer will have 3 neurons (one per class) and use softmax.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
def one_hot(labels, n_classes=3):    encoded =
np.zeros((labels.shape[0], n_classes))
encoded[np.arange(labels.shape[0]), labels] = 1
return encoded

y_train_oh = one_hot(y_train)
y_test_oh = one_hot(y_test)

print("Training samples:", X_train_scaled.shape[0])
print("Test samples:", X_test_scaled.shape[0])
print("\nExample scaled input:", np.round(X_train_scaled[0], 3))
print("Example one-hot label:", y_train_oh[0], "->",
target_names[y_train[0]])
Training
samples:
120
Test
samples:
30
Example      scaled
input:  [-1.722  -
0.332    -1.346  -
1.323]
Example  one-hot
label: [1. 0. 0.]
-> setosa
```

Task 2: Build a Simple Neural Network and Perform Forward Propagation

We'll build a small **feedforward neural network** with one hidden layer:

Input layer (4 neurons)	->	Hidden layer (8 neurons)	->	Output layer (3 neurons)
features		ReLU activation		Softmax
activation				

- **Input layer:** 4 neurons, one for each feature (sepal length/width, petal length/width).
- **Hidden layer:** 8 neurons. Each hidden neuron computes a weighted sum of all 4 inputs plus a bias, then applies an activation function.
- **Output layer:** 3 neurons, one per Iris species. Each output neuron produces a score ("logit") for its class, which we turn into probabilities with softmax.

The math of one layer

For a layer with weight matrix W , bias vector b , and input x :

$$z = xW + b$$

$$a = \text{activation}(z)$$

We do this once to go from input -> hidden, and again to go from hidden -> output.

```
n_inputs = X_train_scaled.shape[1]    # 4 features
n_hidden = 8                          # size of the hidden layer (our
                                     choice)
n_outputs = 3                         # 3 Iris classes

# Randomly initialize weights and biases.
# (In a real trained network these would be learned; here we use
random
# values just to demonstrate how forward propagation works.)
W1 = np.random.randn(n_inputs, n_hidden) * 0.5    # weights:
input -> hidden
b1 = np.zeros((1, n_hidden))                      # bias for
hidden layer

W2 = np.random.randn(n_hidden, n_outputs) * 0.5   # weights: hidden
-> output
b2 = np.zeros((1, n_outputs))                     # bias for
output layer

print("W1 shape (input->hidden):", W1.shape)
print("b1 shape:", b1.shape)
print("W2 shape (hidden->output):", W2.shape)
print("b2 shape:", b2.shape)

W1 shape (input->hidden): (4, 8)
b1 shape: (1, 8)
W2 shape (hidden->output): (8, 3)
b2 shape: (1, 3)
```

Activation functions

We'll use **ReLU** for the hidden layer and **softmax** for the output layer.

```
def relu(z):
    """ReLU activation: keeps positive values, zeroes out
negative ones."""
    return np.maximum(0, z)

def softmax(z):
    """Softmax activation: turns raw scores (logits) into
probabilities
that sum to 1 across each row."""
    # subtract the max for numerical stability (avoids
overflow)
    z_shifted = z - np.max(z, axis=1,
keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)
```

Forward propagation function

This function pushes an input batch `X` through the network: input -> hidden (ReLU) -> output (softmax).

```
def forward_propagation(X, W1, b1, W2, b2):
    # --- Input layer -> Hidden layer ---
    Z1 = np.dot(X, W1) + b1      # weighted sum + bias
    A1 = relu(Z1)                # apply activation function

    # --- Hidden layer -> Output layer ---
    Z2 = np.dot(A1, W2) + b2     # weighted sum + bias
    A2 = softmax(Z2)            # apply activation function ->
    probabilities

    cache = {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}
    return A2, cache

# Run forward propagation on the test set
predicted_probs, cache = forward_propagation(X_test_scaled, W1,
b1, W2, b2)

print("Output shape:", predicted_probs.shape)  # (n_samples, 3
classes)
print("\nFirst 5 predicted probability rows (rows sum to 1):")
print(np.round(predicted_probs[:5], 3))

Output shape: (30, 3) First 5 predicted
probability rows (rows sum to 1):

[0.236      0.289
 0.475]]
```

Task 3: Apply the Activation Function and Display the Predicted Output

The softmax activation already gave us a probability for each of the 3 classes. Now we:

1. Pick the class with the **highest probability** as the prediction (`argmax`).
2. Compare predictions to the true labels.
3. Display the results in a readable table.

Note: Because the weights `W1/W2` above are randomly initialized (not trained), accuracy will be roughly what you'd expect from random guessing (~33%) for 3 classes. This is expected — the point of this task is to understand forward propagation, not to train an accurate model.

```
predicted_classes = np.argmax(predicted_probs, axis=1)

results_df = pd.DataFrame({
    "True Species": [target_names[i] for i in y_test],
```

```

    "Predicted Species": [target_names[i] for i in
predicted_classes],
    "Prob. Setosa": np.round(predicted_probs[:, 0], 3),
    "Prob. Versicolor": np.round(predicted_probs[:, 1], 3),
    "Prob. Virginica": np.round(predicted_probs[:, 2], 3),
    "Correct?": y_test == predicted_classes
})

results_df.head(10)

{"summary": "{\n  \"name\": \"results_df\",\n  \"rows\": 30,\n  \"fields\": [\n    {\n      \"column\": \"True Species\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"setosa\",\n          \"virginica\",\n          \"versicolor\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Predicted Species\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"virginica\",\n          \"versicolor\",\n          \"setosa\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Prob. Setosa\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.13342684508188654,\n        \"min\": 0.168,\n        \"max\": 0.652,\n        \"num_unique_values\": 29,\n        \"samples\": [\n          0.328,\n          0.181,\n          0.299\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Prob. Versicolor\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.04537482465689182,\n        \"min\": 0.244,\n        \"max\": 0.407,\n        \"num_unique_values\": 29,\n        \"samples\": [\n          0.365,\n          0.306,\n          0.391\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Prob. Virginica\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.14979328284741908,\n        \"min\": 0.06,\n        \"max\": 0.588,\n        \"num_unique_values\": 29,\n        \"samples\": [\n          0.307,\n          0.097,\n          0.31\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Correct?\",\n      \"properties\": {\n        \"dtype\": \"boolean\",

```

```

\"num_unique_values\": 2,\ n          \"samples\": [\n
true,\n          false\n          ],\ n          \"semantic_type\":
\\\", \n          \"description\": \\\"\\\" \n          } \n
]\n} \", \"type\": \"dataframe\", \"variable_name\": \"results_df\"}

accuracy = np.mean(predicted_classes == y_test)
print(f\"Accuracy with random (untrained) weights: {accuracy *
100:.2f}
%\")
print(\"This is close to chance level (~33%) since the network
hasn't learned anything yet.\")

```

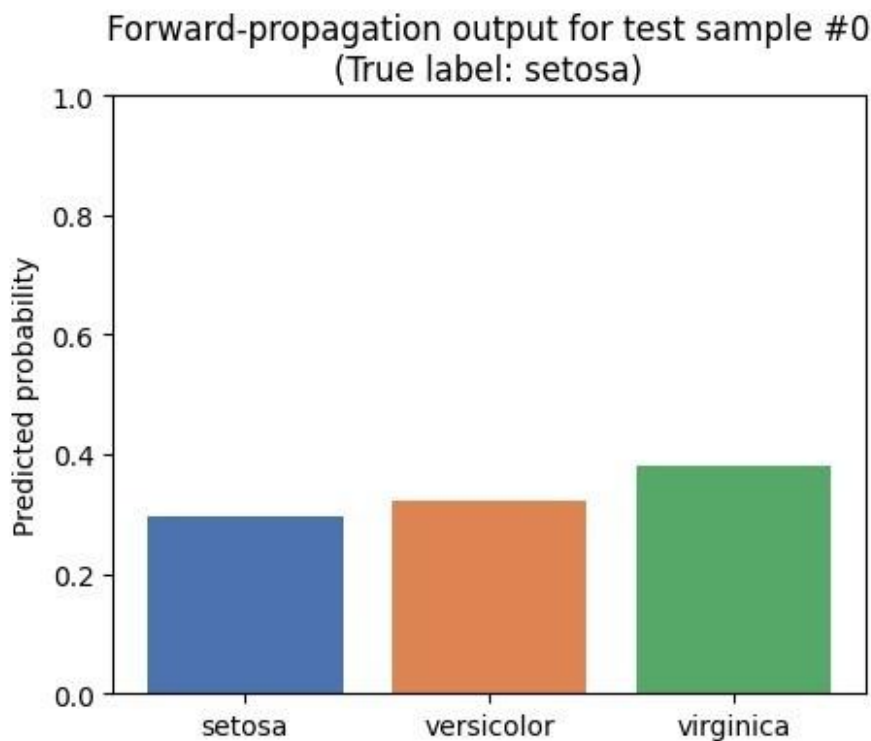
Accuracy with random (untrained) weights: 10.00%

This is close to chance level (~33%) since the network hasn't learned anything yet.

Visualize the predicted probabilities for one sample

```
sample_idx = 0
sample_probs = predicted_probs[sample_idx]

plt.figure(figsize=(5, 4))
plt.bar(target_names, sample_probs, color=['#4C72B0', '#DD8452',
'#55A868'])
plt.ylabel("Predicted probability")
plt.title(f"Forward-propagation output for test sample\n#{sample_idx}\n"
          f"(True label: {target_names[y_test[sample_idx]]})")
plt.ylim(0, 1)
plt.show()
```



Summary

- **Task 1:** Loaded the Iris dataset, explored it, standardized the features, split into train/test sets, and one-hot encoded the labels.
- **Task 2:** Built a simple neural network (4 -> 8 -> 3 neurons) from scratch with NumPy and implemented forward propagation: $Z = XW + b$, $A = \text{activation}(Z)$, applied twice (input -> hidden, hidden -> output).
- **Task 3:** Applied **ReLU** in the hidden layer and **softmax** in the output layer, then used `argmax` to turn the predicted probabilities into a final class prediction, and displayed the results in a table.

